

OPERATING SYSTEMS – USA20302J

Practical Record Work

Done and submitted by

Name

Registration Id.

In Partial Fulfillment of the Requirements for the Degree of B.Sc.

Computer Science



SRM

INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956

DEPARTMENT OF COMPUTER SCIENCE COLLEGE

OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, KATTANKULATHUR - 603203

July-Dec, 2023

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR– 603203



SRM

INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956

BONAFIDE CERTIFICATE

Certified that this record book is a bonafide practical work done by

Name

Registration Id.

in the Computer Science laboratory during academic year 2023-2024 and submitted for the practical examination of III semester course, Operating Systems (USA20302J) held on

_____ **in partial fulfillment of the requirements for the degree of B.Sc. Computer Science**

Course Tutor

HoD

Examiners 1.

2.

INDEX

Ex. No.	Date of Completion	Title	Page No.	Signature of the Course Tutor
1		Virtual Memory Details		
2		Memory Management With Fixed Partitioning Technique		
3		Memory Management With Variable Partitioning Technique		
4		Available Processors in JVM		
5		First-Come, First-Served (FCFS) scheduling algorithm		
6		Shortest Job First (SJF) scheduling algorithm		
7		Priority scheduling algorithm		
8		Round Robin(RR) scheduling algorithm		

Memory Management

Protection and sharing of memory are two important memory management functions. Memory management is primarily concerned with the allocation of main memory to the requesting processes. Memory allocation classified into two types:

1. Contiguous memory allocation
2. Non-contiguous memory allocation

The contiguous memory allocation further classified as:

1. Fixed partition allocation
2. Variable partition allocation

Fixed partition allocation:

In Fixed partition allocation, divide the memory into several fixed size partitions where each partition will accommodate only one program for execution. When a program terminates, that partition is freed for another program waiting in the queue.

Disadvantage: Wastage of memory – Internal fragmentation

Variable partition allocation:

This technique is called dynamic partitioning. Here, OS has to decide about the partition size, everytime a new process is chosen by long-term scheduler. When a process terminates or is swapped out then the memory manager can return the vacated space to the pool of free memory area which partition allocation are made.

Disadvantage: External fragmentation

Ex. No. : 1

Date:

Virtual Memory Details

Aim: To write a Java program to find the maximum and free memory available in the JVM.

Algorithm:

1. Open the editor
2. Import the Runtime class
3. Within the main()
 - a. Create an object for the Runtime class
 - b. Find the total, maximum, and free memory in the JVM using the object
 - c. Do the external garbage collection
 - d. Find the available free memory
4. Save the file with the .java extension
5. Compile and execute the program
6. Close the editor

Program:

```
                // Memory Details
import java.lang.Runtime.*;
class MemoryDetails
{
    public static void main(String []args)
    {
        Runtime r = Runtime.getRuntime();
        System.out.println("Current Runtime: "+r);
        System.out.println("Total virtual memory: "+r.totalMemory());
        System.out.println("Maximum memory will be used by JVM: "+r.maxMemory());
        System.out.println("Available free memory in the JVM: "+r.freeMemory());
        r.gc();
        System.out.println("Available free memory in the JVM after garbage collection:
"+r.freeMemory());
    }
}
```

Output:

```
/*  
F:\OSprogram>javac MemoryDetails.java  
  
F:\OSprogram>java MemoryDetails  
  
Current Runtime: java.lang.Runtime@15db9742  
Total virtual memory: 126877696  
Maximum memory will be used by JVM: 1857028096  
Available free memory in the JVM: 125535480  
Available free memory in the JVM after garbage collection: 125923336  
*/
```

Result:

Thus the program was executed and verified successfully.

Ex. No. : 2
Date:

Memory Management With Fixed Partitioning Technique

Aim: To write a Java program to simulate the fixed partitioning technique.

Algorithm:

1. Open the editor
2. Import the Scanner class
3. Within the main()
 - a. Read the available memory, block size, number of processes, and required memory of the processes
 - b. For each process
 - i. Allocate the block of memory if the required memory is less than the block size and calculate the fragmented memory
 - ii. Otherwise, skip the allocation
4. Save the file with the .java extension
5. Compile and execute the program
6. Close the editor

Program:

```
// Fixed Memory Partitioning
// ~~~~~

import java.util.Scanner;

class FixedPartitioning
{
    public static void main(String []args)
    {
        int pm[]=new int[10];
        int apc=0,tif=0,j;

        Scanner s=new Scanner(System.in);

        System.out.print("Enter the total available memory (in bytes): ");
        int tm=s.nextInt();

        System.out.print("Enter the block size: ");
        int bs=s.nextInt();
```

```

        int nb=tm/bs;
        System.out.print("Enter the number of processes: ");
        int np=s.nextInt();
        for(int i=1;i<=np;i++)
        {
            System.out.print("Enter the memory required for Process"+i+": ");
            pm[i]=s.nextInt();
        }
        System.out.print("\n\nP_ID\tMemory Required\tAllocated\tInternal
Fragmentation");
        for(j=1;j<=np && apc<nb;j++)
        {
            System.out.print("\n" + j + "\t\t" + pm[j] + "\t\t");
            if(pm[j]>bs)
                System.out.print("NO\t\t\t---\n");
            else
            {
                int a=bs-pm[j];
                System.out.print("YES\t\t\t" + a + "\n");
                tif=tif+a;
                apc++;
            }
        }
        if(j<np)
            System.out.println("Memory is Full, remaining processes cannot be
accommodated");
        System.out.println("Total Internal Fragmentation =" + tif);
    }
}

```

Output 1:

```
/*
```

```
F:\II CS OS\OSprogram>java FixedPartitioning
```

```
Enter the total available memory (in bytes): 1000
```

```
Enter the block size: 200
```

```
Enter the number of processes: 6
```

```
Enter the memory required for Process1: 189
```

```
Enter the memory required for Process2: 123
```

```
Enter the memory required for Process3: 255
```

```
Enter the memory required for Process4: 167
```

```
Enter the memory required for Process5: 199
```

```
Enter the memory required for Process6: 200
```

P_ID	Memory Required	Allocated	Internal Fragmentation
1	189	YES	11
2	123	YES	77
3	255	NO	---
4	167	YES	33
5	199	YES	1
6	200	YES	0

```
Total Internal Fragmentation =122    */
```

Output 2:

```
/*
```

```
F:\II CS OS\OSprogram>java FixedPartitioning
```

```
Enter the total available memory (in bytes): 1000
```

```
Enter the block size: 250
```

```
Enter the number of processes: 7
```

```
Enter the memory required for Process1: 230
```

```
Enter the memory required for Process2: 300
```

```
Enter the memory required for Process3: 100
```

```
Enter the memory required for Process4: 222
```

```
Enter the memory required for Process5: 80
```

```
Enter the memory required for Process6: 210
```

```
Enter the memory required for Process7: 50
```

P_ID	Memory Required	Allocated	Internal Fragmentation
1	230	YES	20
2	300	NO	---
3	100	YES	150
4	222	YES	28
5	80	YES	170

```
Memory is Full, remaining processes cannot be accommodated
```

```
Total Internal Fragmentation =368
```

```
*/
```

Result:

Thus the program was executed and verified successfully.

Ex. No. : 3
Date:

Memory Management With Variable Partitioning Technique

Aim: To write a Java program to simulate the Variable partitioning technique.

Algorithm:

1. Open the editor
2. Import the Scanner class
3. Within the main()
 - a. Read the available memory and number of processes
 - b. For each process
 - i. Read the required memory
 - ii. Check if memory is available to allocate. If available, allocate otherwise stop the reading process.
 - c. Calculate the external fragmentation
4. Save the file with the .java extension
5. Compile and execute the program
6. Close the editor

Program:

```
// Variable Memory Partitioning
// ~~~~~

import java.util.Scanner;

class VariablePartitioning
{
    public static void main(String []args)
    {
        int pm[]=new int[10];
        int i,j;
        Scanner s=new Scanner(System.in);
        System.out.print("Enter the total available memory (in bytes): ");
        int tm=s.nextInt();
        int temp=tm;
        System.out.print("Enter the number of processes: ");
```

```

int np=s.nextInt();
for(i=1;i<=np;i++)
{
    System.out.print("Enter the memory required for Process"+i+": ");
    pm[i]=s.nextInt();
    if(temp>=pm[i])
    {
        temp-=pm[i];
    }
    else
    {
        break;
    }
}
if (i<np)
    System.out.println(" Memory is full!!! Sorry process"+i+" can't be
accommodated");
System.out.print("\n\nP_ID\t Allocated Memory(in bytes)");
for(j=1;j<i;j++)
{
    System.out.println("\n\t "+ j+" \t\t"+ pm[j]);
}
System.out.println("Total External Fragmentation =" +temp);
}
}

```

Output :

```
/*  
F:\II CS OS\OSprogram>javac VariablePartitioning.java  
  
F:\II CS OS\OSprogram>java VariablePartitioning  
Enter the total available memory (in bytes): 1000  
Enter the number of processes: 7  
Enter the memory required for Process1: 250  
Enter the memory required for Process2: 150  
Enter the memory required for Process3: 300  
Enter the memory required for Process4: 180  
Enter the memory required for Process5: 300  
Memory is full!!! Sorry process5 can't be accommodated
```

P_ID	Allocated Memory(in bytes)
1	250
2	150
3	300
4	180

```
Total External Fragmentation =120    */
```

Result:

Thus the program was executed and verified successfully.

CPU Scheduling Algorithms

Scheduling processes/work is to finish the work on time . Below are different times with respect to a process.

Arrival Time: Time at which the process arrives in the ready queue.

Completion Time: Time at which process completes its execution.

Burst Time : Time required by a process for CPU execution.

Turn Around Time: Time Difference between completion time and arrival time.

$$\text{Turn Around Time} = \text{Completion Time} - \text{Arrival Time}$$

Waiting Time(W.T): Time Difference between turn around time and burst time.

$$\text{Waiting Time} = \text{Turn Around Time} - \text{Burst Time.}$$

Objectives of Process Scheduling Algorithm

- Max CPU utilization [Keep CPU as busy as possible]
- Fair allocation of CPU.
- Max throughput [Number of processes that complete their execution per time unit]
- Min turnaround time [Time taken by a process to finish execution]
- Min waiting time [Time a process waits in ready queue]
- Min response time [Time when a process produces first response]

Types of Scheduling Algorithms

1. First-Come, First-Served (FCFS)

- It is the simplest of all the scheduling algorithms.
- The basic principle of this algorithm is to allocate the CPU in the order in which the processes arrive.
- It's implementation involves a ready queue that works in First-in First-out order.
- FCFS CPU scheduling algorithm is non-preemptive
- This usually results in poor performance.

2. Shortest Job First (SJF)

- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processer should know in advance how much time process will take.

3. Priority scheduling

- A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
- Equal-priority processes are scheduled in FCFS order.
- This is a non-preemptive, pre-emptive scheduling algorithm.
- Here starvation of process is possible.

4. Round Robin (RR)

- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fix time to execute, it is called a quantum.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted processes.

Ex. No. : 4

Date:

Available Processors in JVM

Aim: To write a Java program to find the number of processors available in JVM.

Algorithm:

1. Open the editor
2. Import the Runtime class
3. Within the main()
 - a. Create an object for the Runtime class
 - b. Find the available processor using the object
4. Save the file with the .java extension
5. Compile and execute the program
6. Close the editor

Program:

```
// Available Processors in JVM
import java.lang.Runtime.*;
class ProcessInformation
{
    public static void main(String []args) throws Exception
    {
        Runtime r = Runtime.getRuntime();
        System.out.println("Current Runtime: "+r);
        System.out.println("Total number of processors in JVM: "+r.availableProcessors());
    }
}
```

Output :

```
/* F:\II CS OS\OSprogram>javac ProcessInformation.java
F:\II CS OS\OSprogram>java ProcessInformation
Current Runtime: java.lang.Runtime@15db9742
Total number of processors in JVM: 4
*/
```

Result:

Thus the program was executed and verified successfully.

Ex. No. : 5

Date:

First-Come, First-Served (FCFS) scheduling algorithm

Aim: To simulate the First-Come, First-Served (FCFS) scheduling algorithm.

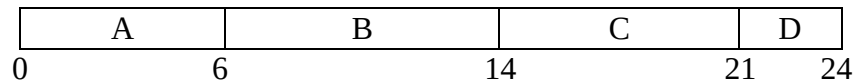
Problem Description:

Consider four processes – A, B, C, and D with their burst-times and arrival times given below:

Process	CPU Burst Time (ms)	Arrival Time(ms)
A	6	0
B	8	0
C	7	0
D	3	0

Manual Calculation:

The GANTT chart is –



Turn around time (TAT) = (Process finish time – Process arrival time)

∴ A's turn around time = $6 - 0 = 6$

B's turn around time = $14 - 0 = 14$

C's turn around time = $21 - 0 = 21$

D's turn around time = $24 - 0 = 24$

∴ Average turn around time (ATAT) = $(6+14+21+24)/4 = 65/4 = 16.25\text{ms}$

Waiting time (WT) = (Turn around time – Processing/Burst time)

∴ A's waiting time = $6 - 6 = 0$

B's waiting time = $14 - 8 = 6$

C's waiting time = $21 - 7 = 14$

D's waiting time = $24 - 3 = 21$

∴ Average waiting time (AWT) = $(0+6+14+21)/4 = 41/4 = 10.25\text{ms}$

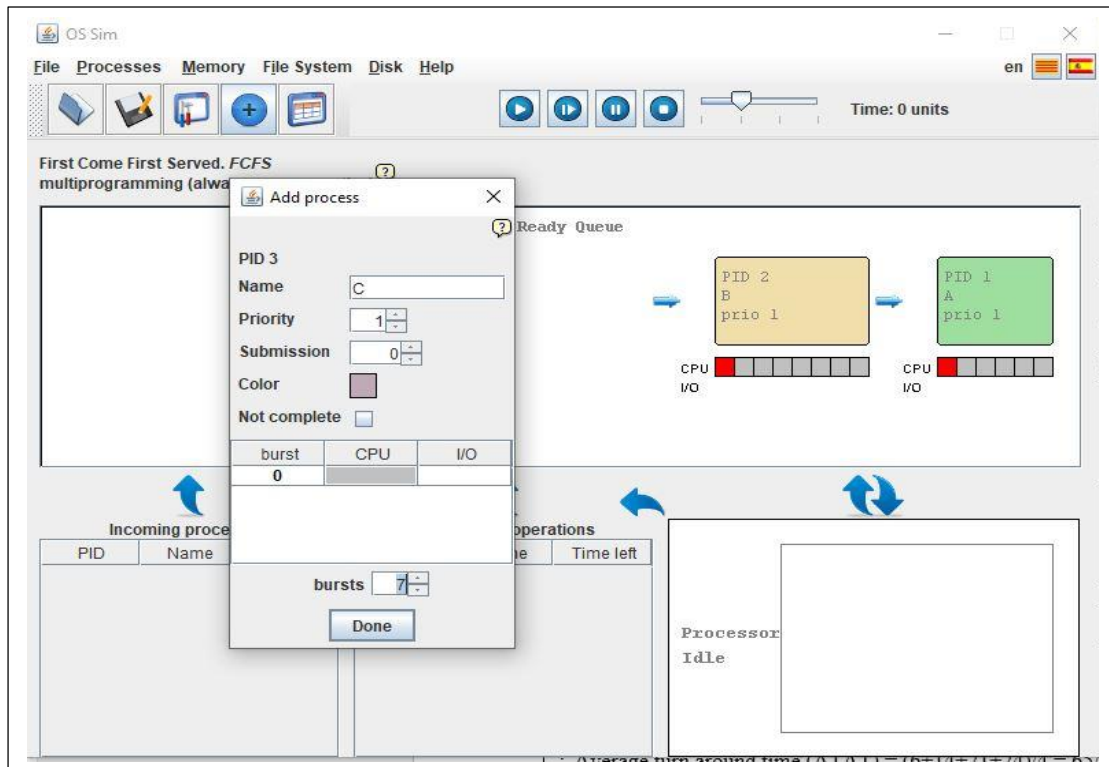
Simulation:

Algorithm:

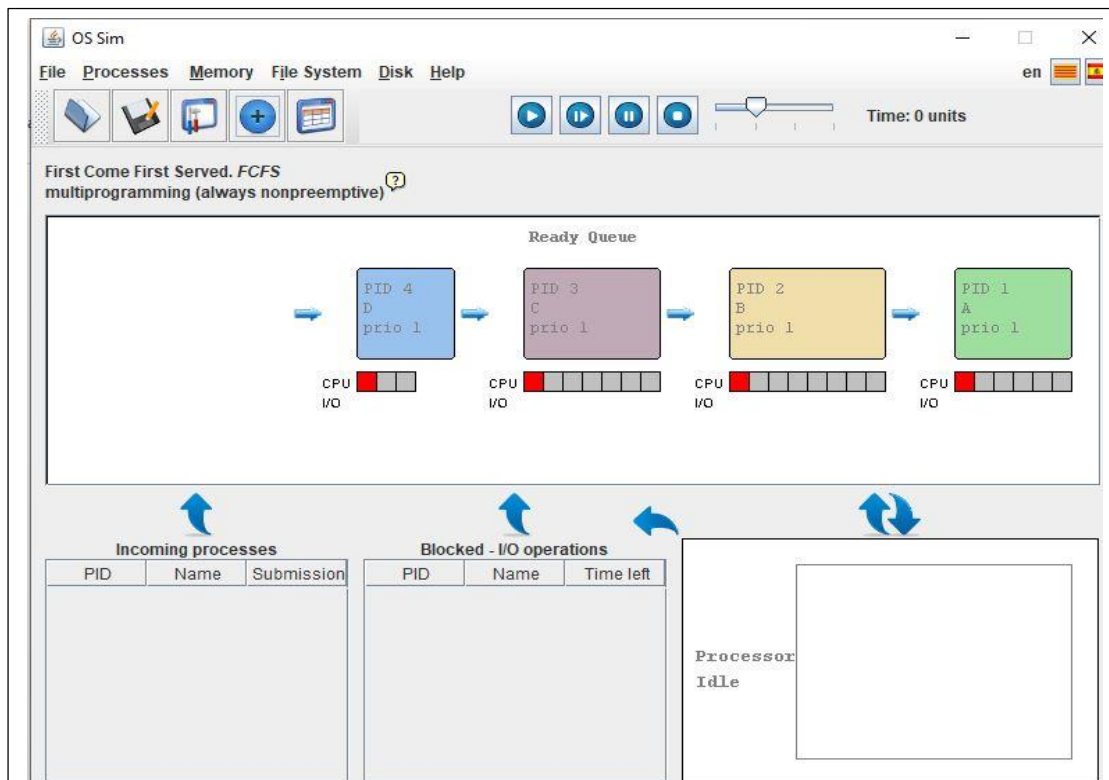
1. Start the ossim simulator
2. Select process scheduling from processes menu
3. Select FCFS algorithm
4. For each process-
 - a. Assign the process name
 - b. Assign the burst time
5. Run the simulator
6. Generate the report
7. Close the simulator

Sample simulation steps:

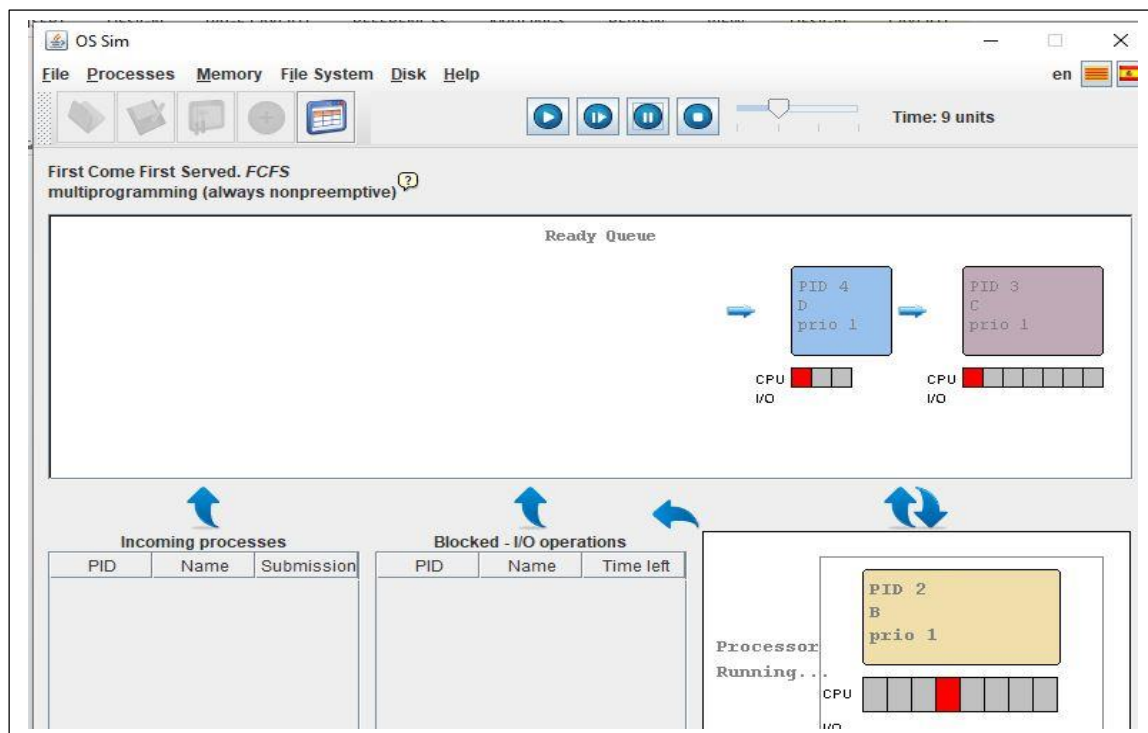
1. Process information



2. Ready queue



3. Scheduling the process



4. Scheduling Report

OS Sim interface showing the Process Scheduling Information report. The report displays efficiency, throughput, and average turnaround, waiting, and response times for the four processes.

Process Scheduling Information

Efficiency (%) 1.00

Throughput (processes/time unit) 0.17

Avg. Turnaround Time (time) 16.25

Avg. Waiting Time (time) 10.25

Avg. Response Time (time) 10.25

PID	Name	Priority	Submissi...	Periodic	CPU	Response	Waiting	Turnarou...	% CPU	% IO
1	A	1	0	-	6	0	0	6	1.0	0.0
2	B	1	0	-	8	6	6	14	0.571428...	0.0
3	C	1	0	-	7	14	14	21	0.333333...	0.0
4	D	1	0	-	3	21	21	24	0.125	0.0

Result:

Thus the simulation was executed and verified successfully.

Ex. No. : 6

Date:

Shortest Job First (SJF) scheduling algorithm

Aim: To simulate the Shortest Job First (SJF) scheduling algorithm.

Problem Description:

Consider four processes – A, B, C, and D with their burst-times and arrival times given below:

Process	CPU Burst Time (ms)	Arrival Time(ms)
A	6	0
B	8	0
C	7	0
D	3	0

Manual Calculation:

The GANTT chart is –

D	A	C	B	
0	3	9	16	24

Turn around time (TAT) = (Process finish time – Process arrival time)

∴ A's turn around time = $9 - 0 = 9$

B's turn around time = $24 - 0 = 24$

C's turn around time = $16 - 0 = 16$

D's turn around time = $3 - 0 = 3$

∴ Average turn around time (ATAT) = $(9+24+16+3)/4 = 52/4=13\text{ms}$

Waiting time (WT) = (Turn around time – Processing/Burst time)

∴ A's waiting time = $9 - 6 = 3$

B's waiting time = $24 - 8 = 16$

C's waiting time = $16 - 7 = 9$

D's waiting time = $3 - 3 = 0$

∴ Average waiting time (AWT) = $(3+16+9+0)/4 = 28/4= 7\text{ms}$

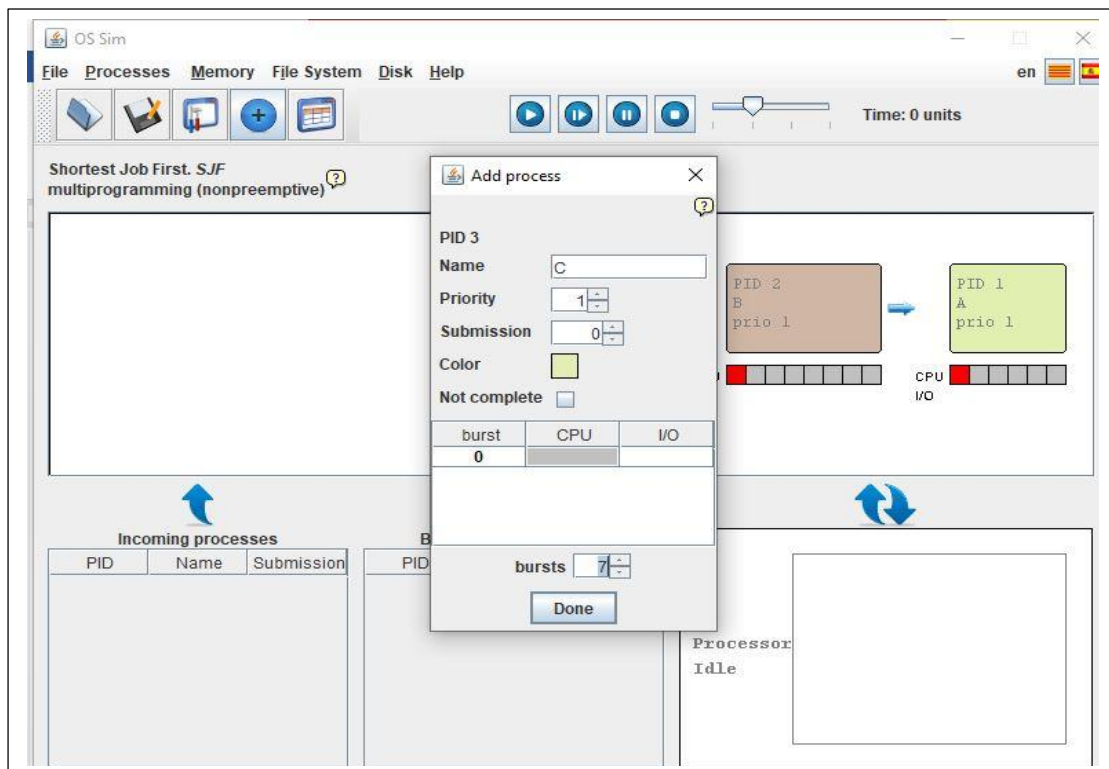
Simulation:

Algorithm:

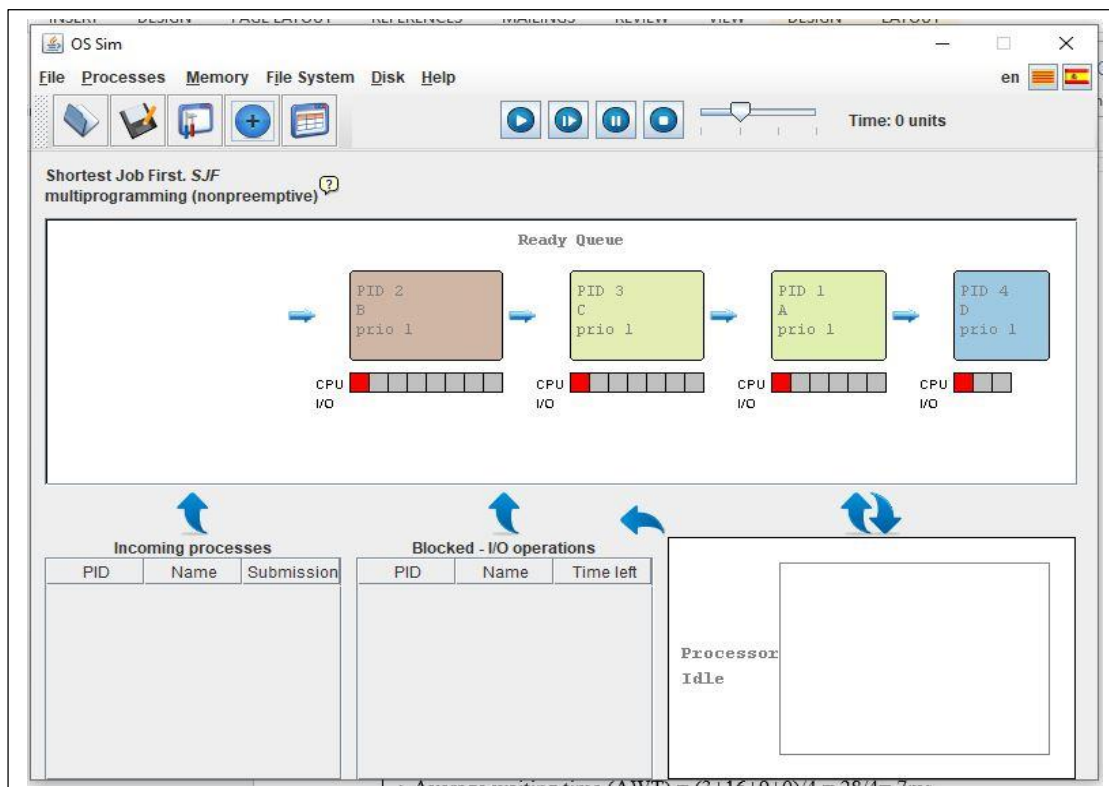
1. Start the ossim simulator
2. Select process scheduling from the processes menu
3. Select SJF algorithm
4. For each process-
 - a. Assign the process name
 - b. Assign the burst time
5. Run the simulator
6. Generate the report
7. Close the simulator

Sample simulation steps:

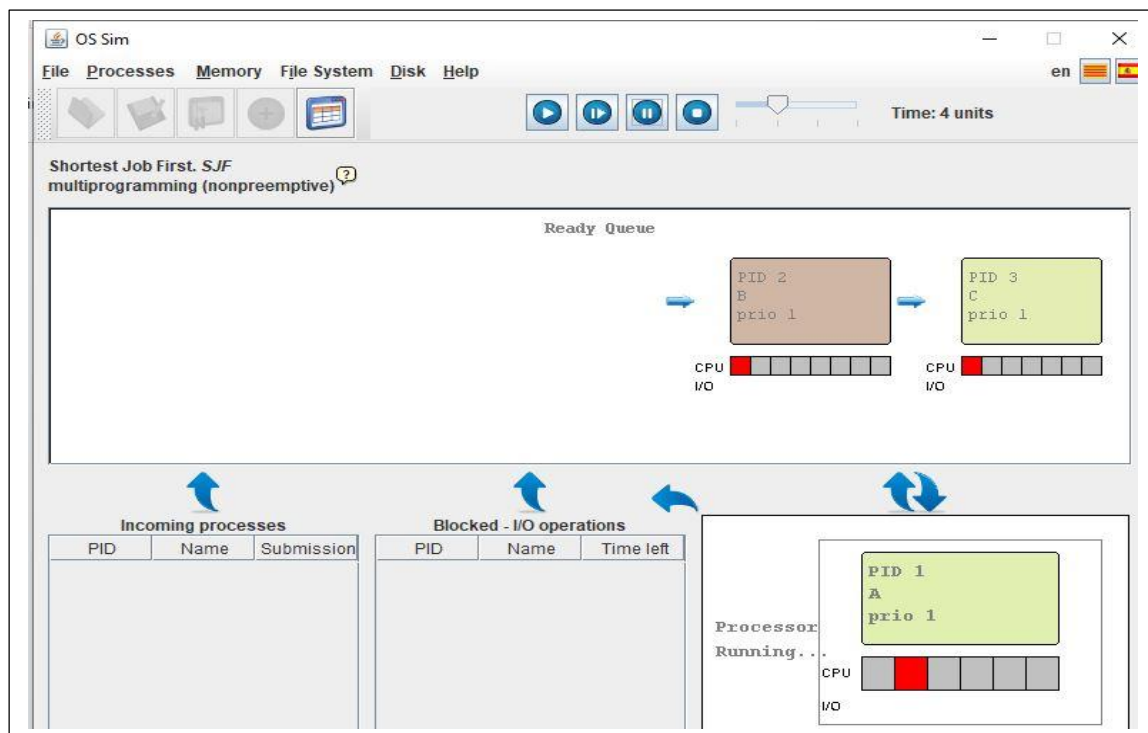
1. Process information



2. Ready queue



3. Scheduling the process



4. Scheduling Report

OS Sim

File Processes Memory File System Disk Help

Time: 24 units

Process Scheduling Information

Efficiency (%) 1.00

Throughput (processes/time unit) 0.17

Avg. Turnaround Time (time) 13.00

Avg. Waiting Time (time) 7.00

Avg. Response Time (time) 7.00

PID	Name	Priority	Submission	Periodic	CPU	Response	Waiting	Turnaround	% CPU	% IO
4	D	1	0	-	3	0	0	3	1.0	0.0
1	A	1	0	-	6	3	3	9	0.6666666...	0.0
3	C	1	0	-	7	9	9	16	0.4375	0.0
2	B	1	0	-	8	16	16	24	0.3333333...	0.0

Result:

Thus the simulation was executed and verified successfully.

Ex. No. : 7**Date:****Priority scheduling algorithm****Aim:** To simulate the Priority scheduling algorithm.**Problem Description:**

Consider the following set of processes, assumed to have arrived at time 0 in the order A, B, C, D, and E, with the length of the CPU burst given in milliseconds. We assume that high numbers represent high priority.

Process	CPU Burst Time (ms)	Priority
A	10	3
B	1	5
C	2	2
D	1	1
E	5	4

Manual Calculation:

The GANTT chart is –

B	E	A	C	D	
0	1	6	16	18	19

Turn around time (TAT) = (Process finish time – Process arrival time)

∴ A's turn around time = $16 - 0 = 16$

B's turn around time = $1 - 0 = 1$

C's turn around time = $18 - 0 = 18$

D's turn around time = $19 - 0 = 19$

E's turn around time = $6 - 0 = 6$

∴ Average turn around time (ATAT) = $(16+1+18+19+6)/5 = 60/5=12\text{ms}$

Waiting time (WT) = (Turn around time – Processing/Burst time)

∴ A's waiting time = $16 - 10 = 6$

B's waiting time = $1 - 1 = 0$

C's waiting time = $18 - 2 = 16$

D's waiting time = $19 - 1 = 18$

E's waiting time = $6 - 5 = 1$

∴ Average waiting time (AWT) = $(6+0+16+18+1)/5 = 41/5 = 8.2\text{ms}$

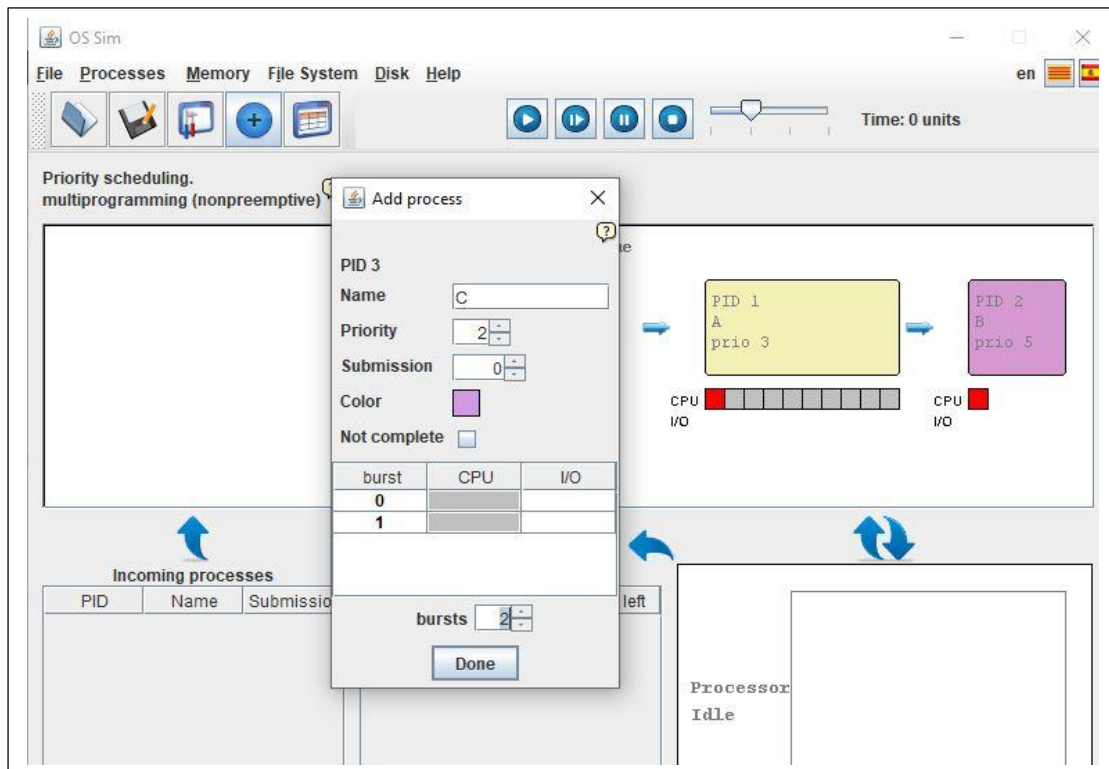
Simulation:

Algorithm:

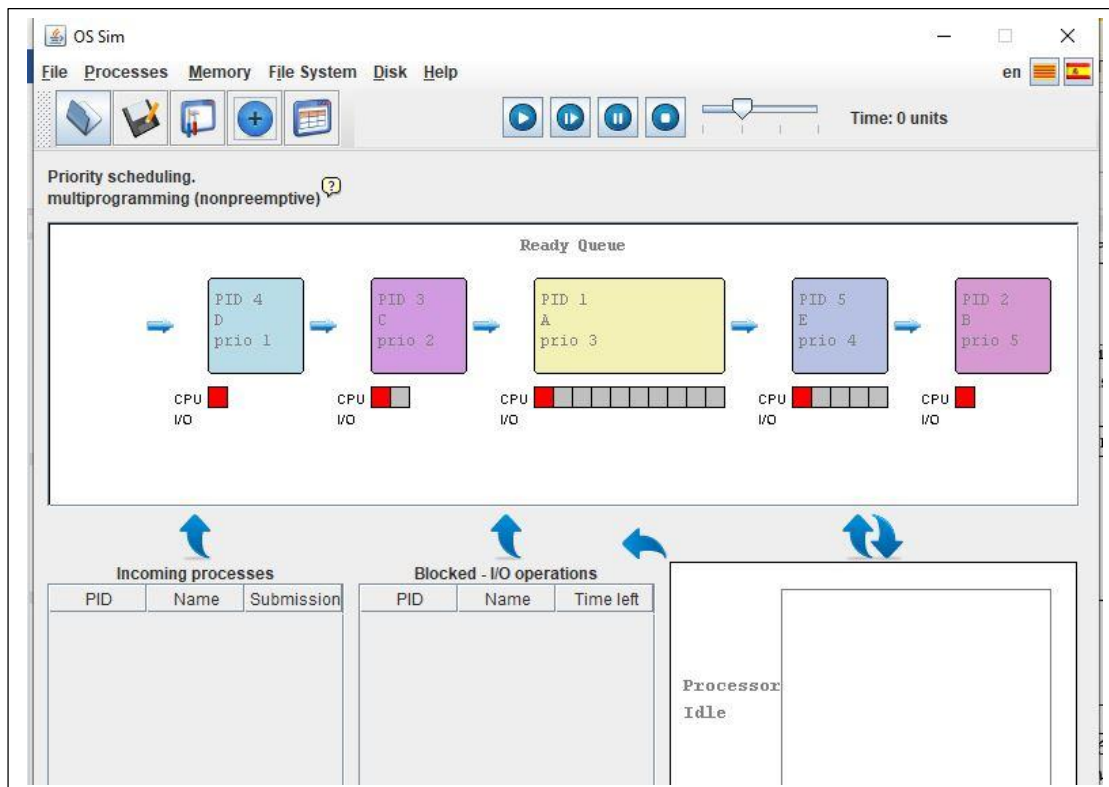
1. Start the ossim simulator
2. Select process scheduling from the processes menu
3. Select Priority scheduling algorithm
4. For each process-
 - a. Assign the process name
 - b. Assign the priority
 - c. Assign the burst time
5. Run the simulator
6. Generate the report
7. Close the simulator

Sample simulation steps:

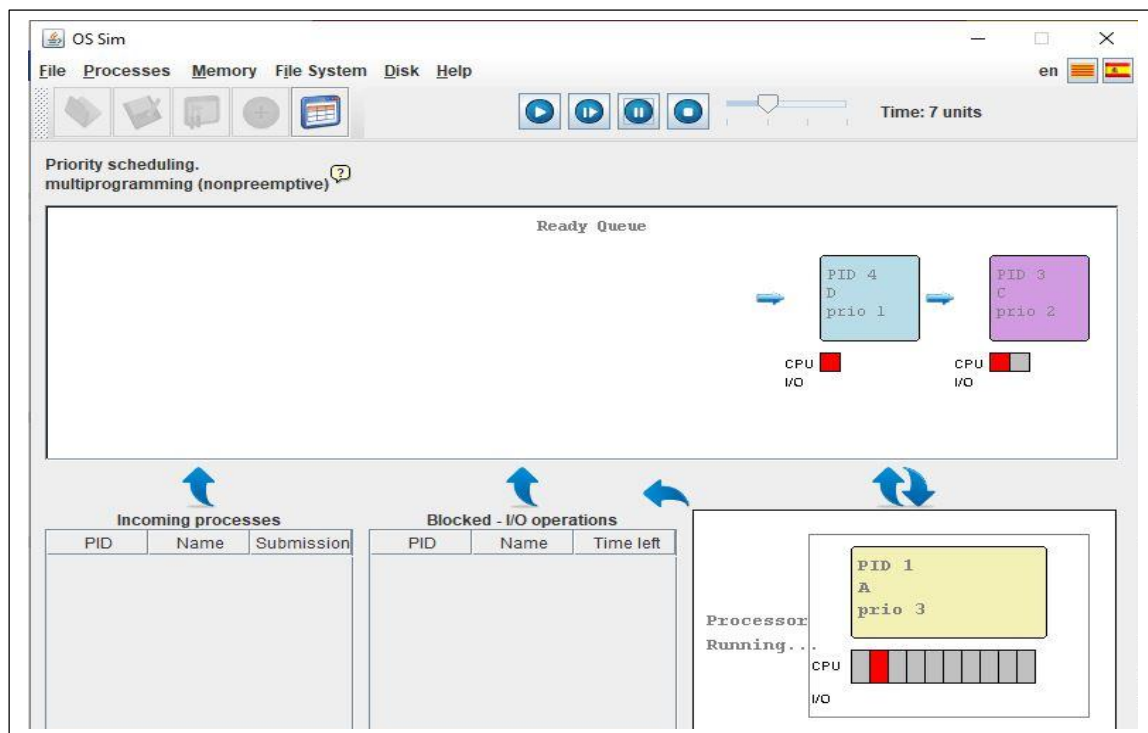
1. Process information



2. Ready queue



3. Scheduling the process



4. Scheduling Report

Process Scheduling Information

Efficiency (%) 1.00

Throughput (processes/time unit) 0.26

Avg. Turnaround Time (time) 12.00

Avg. Waiting Time (time) 8.20

Avg. Response Time (time) 8.20

PID	Name	Priority	Submission	Periodic	CPU	Response	Waiting	Turnaround	% CPU	% IO
2	B	5	0	-	1	0	0	1	1.0	0.0
5	E	4	0	-	5	1	1	6	0.8333333...	0.0
1	A	3	0	-	10	6	6	16	0.625	0.0
3	C	2	0	-	2	16	16	18	0.1111111...	0.0
4	D	1	0	-	1	18	18	19	0.0526315...	0.0

Result:

Thus the simulation was executed and verified successfully.

Ex. No. : 8

Date:

Round Robin(RR) scheduling algorithm

Aim: To simulate the Round Robin(RR) scheduling algorithm.

Problem Description:

Consider the following set of processes — A, B and C that arrive at time 0 and time quantum of 4 milliseconds.. That is,

Process	CPU Burst Time (ms)
A	10
B	3
C	3

Manual Calculation:

The GANTT chart is –

0	A	4	B	7	C	10	A	14	A	16
---	---	---	---	---	---	----	---	----	---	----

Turn around time (TAT) = (Process finish time – Process arrival time)

∴ A's turn around time = 16 – 0 = 16

B's turn around time = 7 – 0 = 7

C's turn around time = 10 – 0 = 10

∴ Average turn around time (ATAT) = $(16+7+10)/3 = 33/3=11\text{ms}$

Waiting time (WT) = (Turn around time – Processing/Burst time)

∴ A's waiting time = 16– 10 = 6

B's waiting time = 7 – 3 = 4

C's waiting time = 10 – 3 = 7

∴ Average waiting time (AWT) = $(6+4+7)/3 = 17/3= 5.66\text{ms}$

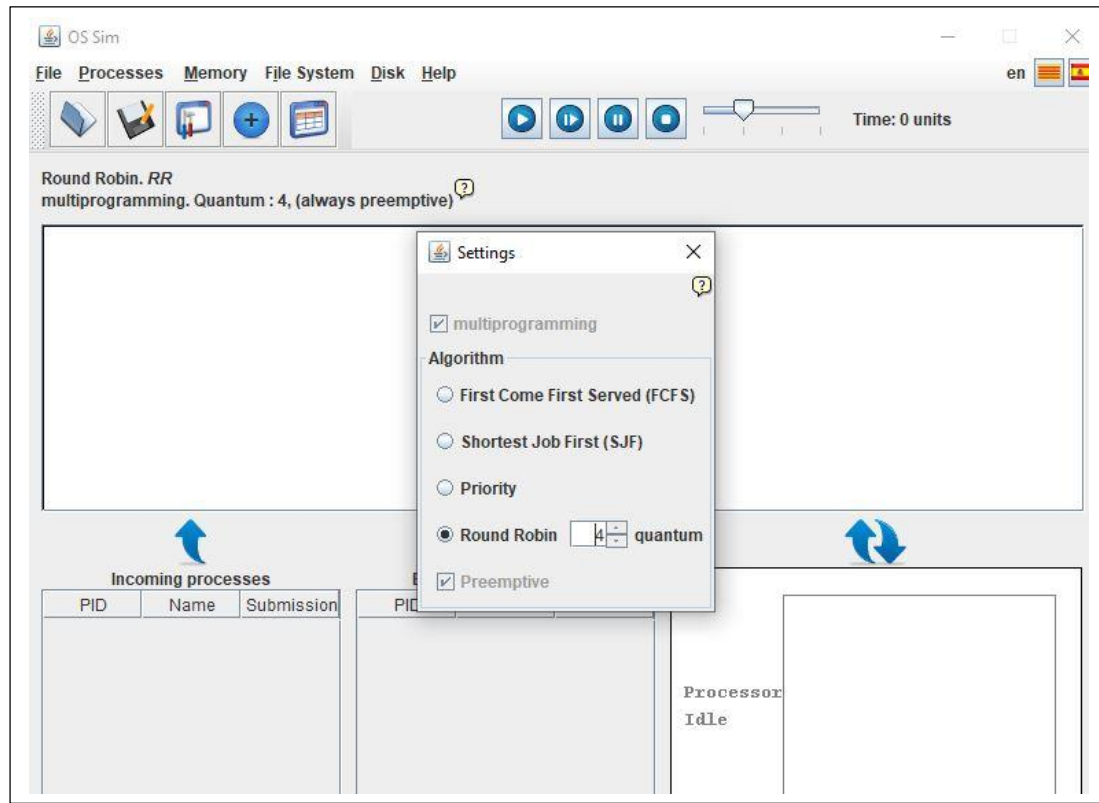
Simulation:

Algorithm:

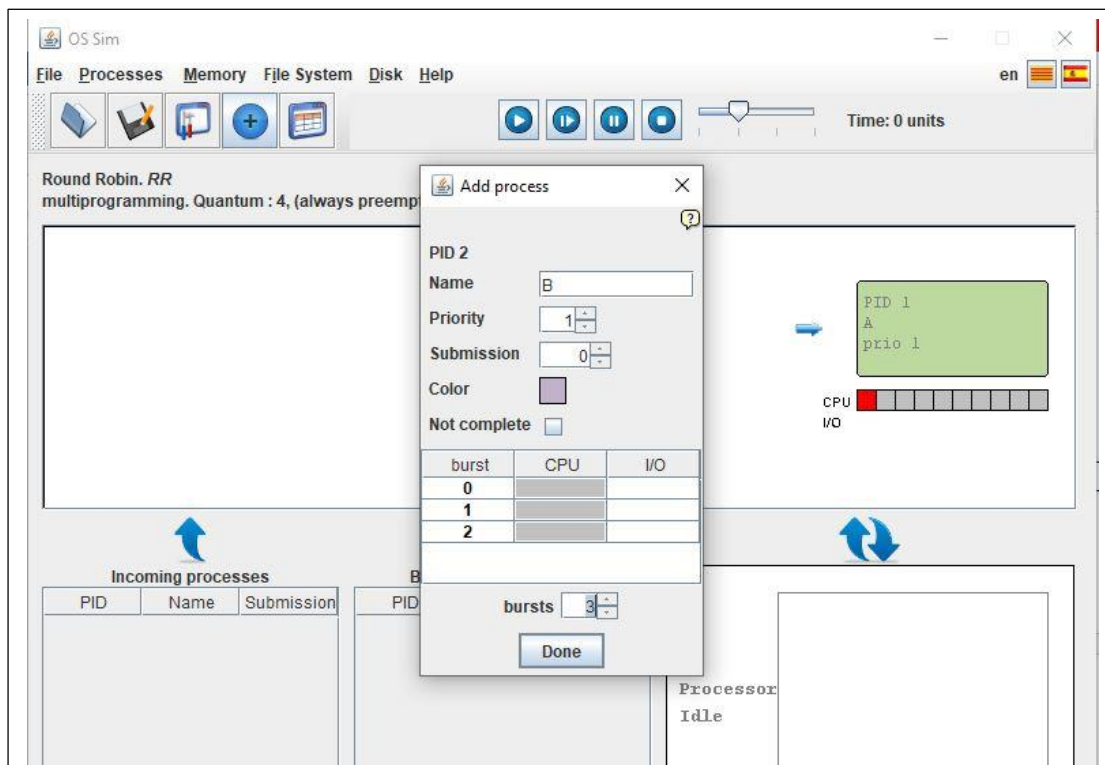
1. Start the ossim simulator
2. Select process scheduling from the processes menu
3. Select RR algorithm
4. For each process-
 - a. Assign the process name
 - b. Assign the burst time
5. Run the simulator
6. Generate the report
7. Close the simulator

Sample simulation steps:

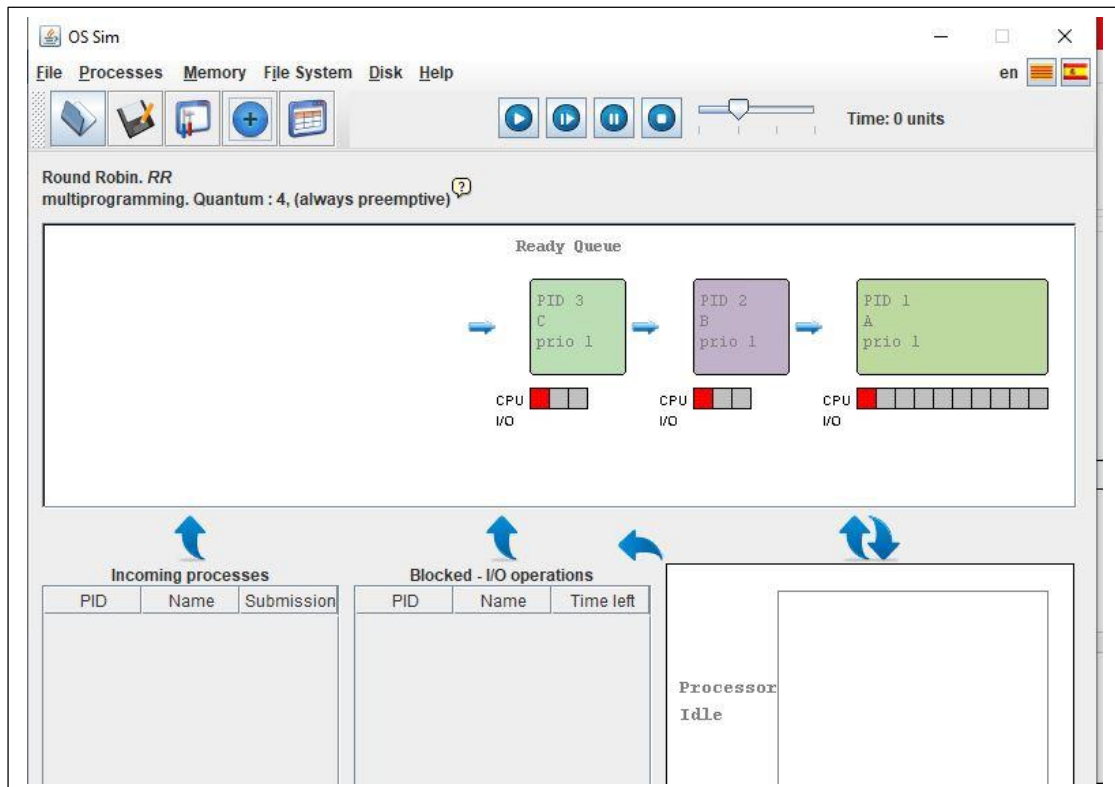
1. Time quantum setting



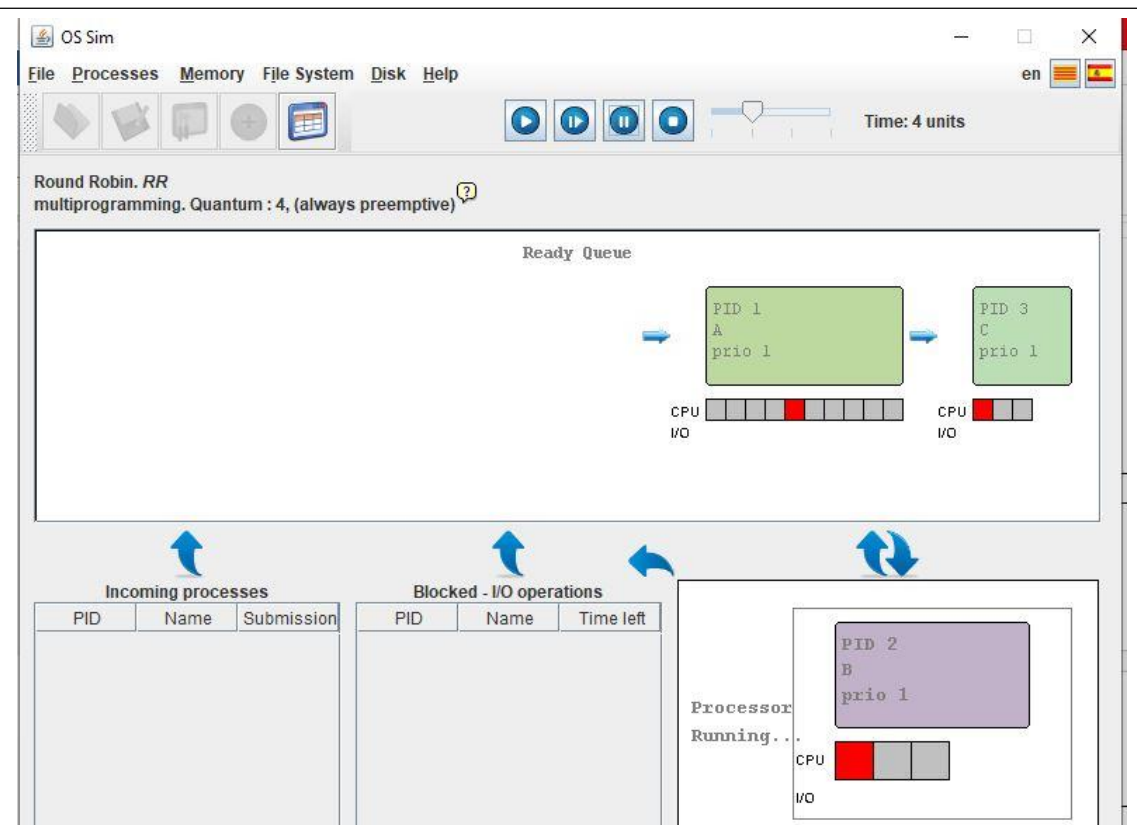
2. Process information



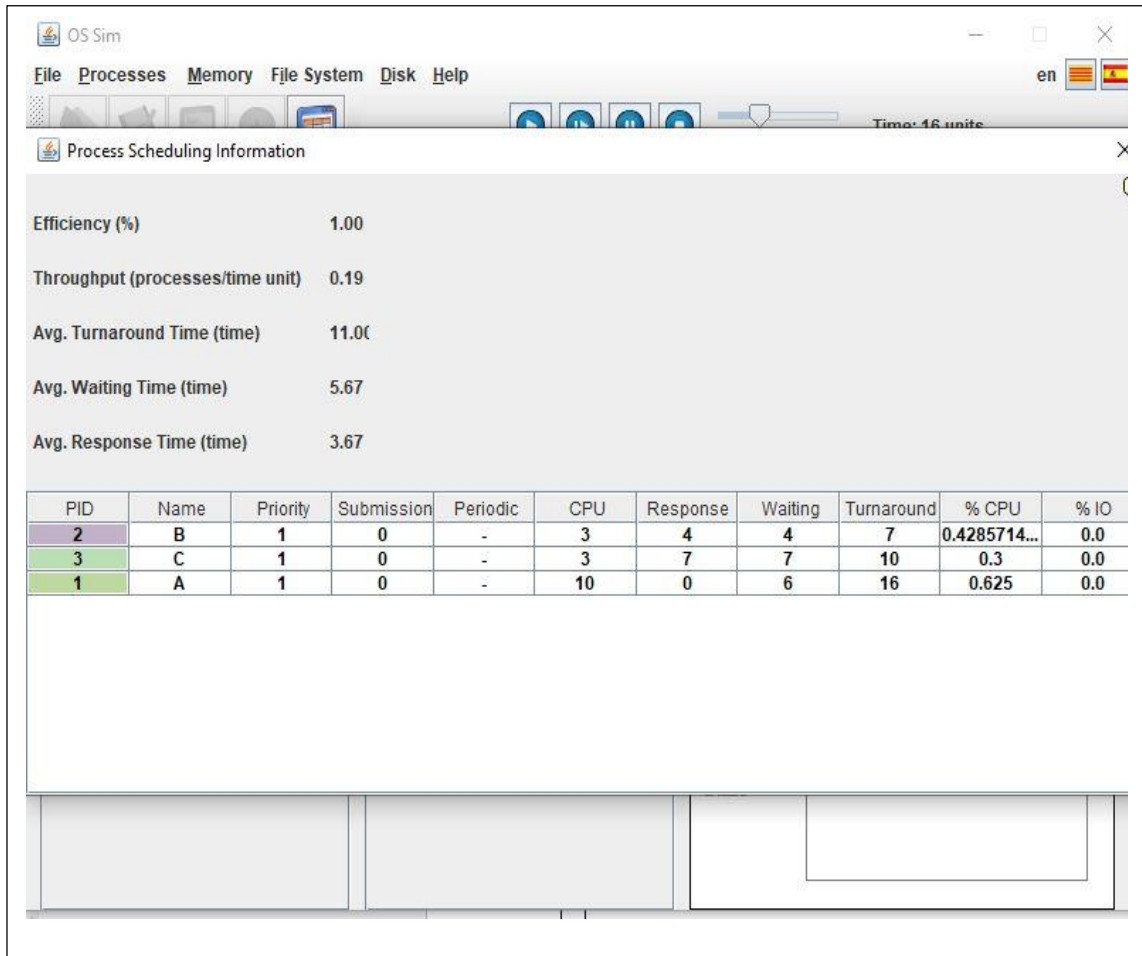
3. Ready queue



4. Scheduling the process



5. Scheduling Report



Result:

Thus the simulation was executed and verified successfully.